

Consumo de servicios RESTful (I)

El objetivo de esta práctica es aprender a invocar operaciones básicas en servicios RESTful, tanto a nivel visual (Postman), como a nivel programático (Spring Boot). También aprenderemos a implementar pruebas automatizadas con JUnit.

Ejercicios

Ejercicio 1. Operaciones de consulta desde Postman

Invoke a las siguientes operaciones de consulta de APIs RESTful públicas haciendo uso de la interfaz gráfica de Postman. Para ello, cree una colección en Postman y añada una petición (request) para cada una de las siguientes operaciones:

- Obtener listado de todos los objetos. <https://restful-api.dev/>
- Información de los últimos lanzamientos al espacio de la empresa SpaceX. <https://github.com/r-spacex/SpaceX-API>
- Obtener listado de personajes de Juego de Tronos. <https://github.com/joakimskoog/AnApiOfIceAndFire/wiki>
- Obtener listado de personajes femeninos de Juego de Tronos. <https://github.com/joakimskoog/AnApiOfIceAndFire/wiki>
- Información sobre el Pokemon con nombre Golem. <https://pokeapi.co/docs/v2#pokemon-section>
- Bicicletas robadas en un radio de 100 millas desde Chicago. https://bikeindex.org/documentation/api_v3
- Commits del repositorio de GitHub con nombre spring-framework y propietario spring-projects. <https://docs.github.com/en/rest/commits/commits>
- Issues cerradas (state=closed) del proyecto de GitLab con identificador 4207231. <https://docs.gitlab.com/ee/api/issues.html#list-project-issues>

Ejercicio 2. Creación de proyecto Spring Boot

Cree un proyecto base de Spring Boot y ábralo desde IntelliJ. Para ello, tiene dos opciones. La primera opción consiste en usar el asistente integrado en IntelliJ desde Nuevo proyecto → Spring Initializr. La segunda opción es hacer uso de la aplicación web <https://start.spring.io/> e importar el código generado como un nuevo proyecto. En ambos casos, use los siguientes valores de configuración:

- Project: Maven Project
- Language: Java
- Spring Boot: 3.5.12 (o similar)
- Packaging: Jar
- Java: 21

- Dependencies: Spring Web, Spring Data JPA, H2 Database, Validation.

The screenshot shows the Spring Initializr web application interface. It is divided into two main sections: Project Metadata and Dependencies.

Project Metadata:

- Project:** Radio buttons for Gradle - Groovy, Gradle - Kotlin, **Java** (selected), Kotlin, and Groovy.
- Language:** Radio buttons for **Java** (selected), Kotlin, and Groovy.
- Spring Boot:** Radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (M2), 4.0.4 (SNAPSHOT), 4.0.3, **3.5.12 (SNAPSHOT)** (selected), and 3.5.11.
- Project Metadata:**
 - Group:** aiss
 - Artifact:** restclient
 - Name:** restclient
 - Description:** My first REST client
 - Package name:** aiss.restclient
 - Packaging:** **Jar** (selected), War
 - Configuration:** **Properties** (selected), YAML
 - Java:** Radio buttons for 25, **21** (selected), and 17.

Dependencies:

- Spring Web:** **WEB** (selected). Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Spring Data JPA:** **SQL** (selected). Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
- H2 Database:** **SQL** (selected). Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.
- Validation:** **IO** (selected). Bean Validation with Hibernate validator.

At the bottom, there are buttons for **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and a menu icon (three dots).

Figura 1. Spring initializr.

Ejercicio 3. Operación de consulta básica con Spring

Implemente en el proyecto anterior el código necesario para obtener un listado de todos los objetos desde <https://restful-api.dev/>. Para ello, siga los siguientes pasos:

- Generar las clases POJO para almacenar los datos recibidos desde el servicio en formato JSON haciendo uso de <https://www.jsonschema2pojo.org/>
- Generar una clase de servicio para invocar al servicio y devolver el listado de objetos.
- Implemente una clase de pruebas Junit para probar la funcionalidad implementada.

Ejercicio 4. Operación de consulta básica con Spring (II)

Extienda el código anterior para implementar una nueva operación que permita consultar los datos de un objeto concreto dado su identificador. Implemente la prueba unitaria correspondiente.

Ejercicio 5. Operación de consulta con Spring

Implemente el código necesario para obtener el listado de los commits de un repositorio de GitHub.